

StegoStamp Image Watermarking

Eugene Francisco

June 2026

1 Overview

Watermarking refers to the process of embedding information in generated data so as to label that data as having been generated by a model. This paper introduces StegoStamp, a novel way to watermark images, robust to common image transformations and lossy compression and virtually undetectable to the human eye. Inspired by Bui et al.¹, we train a *message encoder* to embed hidden bit sequences in the latent representation of images from a fixed autoencoder; alongside this, we train a *secret decoder* to recover messages from possibly corrupted images. We take advantage of convolutional autoencoders to allow for the spatial embedding of these watermarks; alongside this, we use techniques from Tancik et al.² to make these watermarkers robust to spatial transformations like cropping and rotating, as well as regular image transformations like noising, blurring, and JPEG.

2 Related Work

Discuss Bui et al.³ and Tancik et al.⁴ and HiDDeN.

3 Methods

Watermarking derives from the much older technique of steganography, where the challenge is to conceal messages within another medium. Stated concretely, the goal of image watermarking is to take a so-called *cover image* c and message m and generate a *stego image* $\tilde{c}$ in which m is somehow embedded; later on, a verifier should be able to take $\tilde{c}$ (which has potentially been compressed or transformed in some way) and recover a message $\hat{m}$ which should be similar to m .

3.1 Architecture

StegoStamp trains two neural networks:

- (i) A message encoder $F_\theta : \{0,1\}^\ell \rightarrow \mathbb{R}^{P \times P \times C}$ which is used to encode messages and generate watermarked images.
- (ii) A secret decoder $D_\varphi : \mathbb{R}^{H \times W \times C} \rightarrow \{0,1\}^\ell$ which is used to recover a message $\hat{m}$ from a (possibly corrupted) $\tilde{c}$.

The message encoder F_θ takes advantage of a fixed image autoencoder to embed messages. Let (E, G) be an image autoencoder. E is an image encoder that sends images $c \in \mathbb{R}^{H \times W \times C}$ to $z = E(G) \in \mathbb{R}^{H' \times W' \times C}$ where H', W' are a constant factor smaller than H and W . And G is an image generator that takes latent variables $z \in \mathbb{R}^{H' \times W' \times C}$ and sends them to $c = G(z) \in \mathbb{R}^{H \times W \times C}$. It is important that (E, G) are convolutional so that they preserve spatial information. (E, G) are fixed in advance and aren't trained.

The networks F_θ and D_φ encode in the following way:

- (i) Message Encoder: given a cover image c of size $H \times W$ and a message m of length ℓ , we first embed c into its latent space to make latent variable $z \leftarrow E(c)$. Assign $\delta \leftarrow F_\theta(m) \in \mathbb{R}^{P \times P \times C}$; this will encode watermarked information. Partition z into patches of size $P \times P \times C$; let $\mathcal{P}$ be the set of these patches.

For each patch $p \in \mathcal{P}$, create the watermarked patch $\tilde{p} \leftarrow p + \delta$. Using the watermarked patches, stitch together a watermarked latent $\tilde{z}$. Finally, return $G(\tilde{z})$ as the watermarked cover. See Algorithm 1.

The neural network F_θ is a very small convolutional neural network implemented in a similar way to Bui et al.

- (ii) Secret Decoder: Given a (possibly corrupted) watermarked image $\tilde{c}$, the secret decoder returns $\hat{m} = D_\varphi(\tilde{c})$. The neural network $D_\varphi(c)$ is implemented as a modified ResNet50 with the final flattening layer replaced by a 1×1 CNN

¹ **TODO** add reference
² **TODO** add reference
³ **TODO** reference
⁴ **TODO** reference

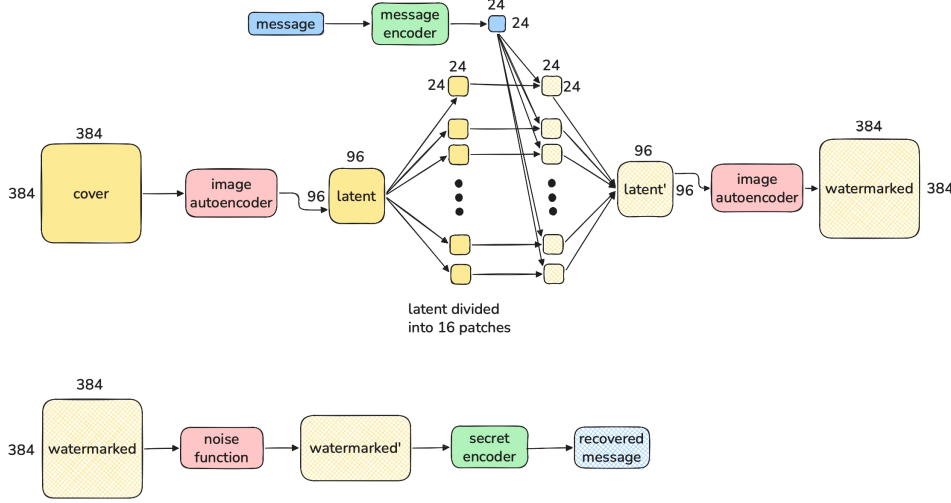


Figure 1: StegoStamp Watermarking and Decoding Architecture. The message encoder refers to F_θ and the secret decoder refers to D_φ .

whose output is ℓ channelled. An average is taken across the width and height dimensions to retrieve an ℓ length vector of logits which are thresholded and returned as $\hat{m}$. See Algorithm 2

This last phase *voting* step is taken from Zhu et al.'s HiDDeN.

Here, θ and φ are trainable parameters; note in particular that the autoencoder (E, G) remain fixed and are not trained.

In this architecture, it is important that the patching and periodic pasting of the latent variable δ happens within the latent space. Watermarking performance worsens significantly if instead we choose to watermark each patch individually and then stitch together the patches in image space. Doing so leads to more obvious stitch marks and much higher quality loss during training.

3.2 Training Details

We use the same training objectives as RoSteALS. F_θ is trained to minimize

$$\mathcal{L}_{\text{quality}} = \mathcal{L}_{\text{LIPS}}(\tilde{c}, c) + \alpha \|\gamma(c) - \gamma(\tilde{c})\|_2^2 \quad (1)$$

where $\mathcal{L}_{\text{LIPS}}$ is the *Learned Perceptual Image Patch Similarity* loss from Zhang et al.⁵ and γ is a differentiable map from RGB to YUV color space. α is a hyperparameter to control objective weight. D_φ is

Algorithm 1 Message Encoder F_θ

Require: cover image $c \in \mathbb{R}^{H \times W \times C}$, message $m \in \{0, 1\}^\ell$

Ensure: stego image $\tilde{c}$

- 1: $z \leftarrow E(c)$
 - 2: $\delta \leftarrow F_\theta(m)$
 - 3: $\mathcal{P} \leftarrow \text{PARTITION}(z) \triangleright \text{into } P \times P \times C \text{ patches}$
 - 4: **for** each patch $p \in \mathcal{P}$ **do**
 - 5: $\tilde{p} \leftarrow p + \delta$
 - 6: **end for**
 - 7: $\tilde{z} \leftarrow \text{STITCH}(\{\tilde{p}\})$
 - 8: **return** $G(\tilde{z})$
-

Algorithm 2 Secret Decoder D_φ

Require: (possibly corrupted) stego image $\tilde{c} \in \mathbb{R}^{H \times W \times C}$

Ensure: recovered message $\hat{m} \in \{0, 1\}^\ell$

- 1: $L \leftarrow \text{MODIFIEDRESNET50}_\varphi(\tilde{c}) \triangleright \text{Last layer is a } 1 \times 1 \times \text{CNN, } \ell \text{ channels. } L \in \mathbb{R}^{H' \times W' \times \ell}$
 - 2: $\bar{L} \leftarrow \text{MEAN}_{H', W'}(L)$
 - 3: $\hat{m} \leftarrow \mathbf{1}[\bar{L} > 0]$
 - 4: **return** $\hat{m}$
-

⁵ **TODO** Add reference

trained to minimize the binary cross entropy between m and $\hat{m}$:

$$\mathcal{L}_{\text{recovery}} = \mathcal{L}_{\text{BCE}}(m, \hat{m}). \quad (2)$$

The combined training objective is then

$$\mathcal{L} = \beta \mathcal{L}_{\text{quality}} + \mathcal{L}_{\text{recovery}} \quad (3)$$

where β is a weighting hyperparameter.

We use curriculum learning to train F_θ and D_φ . Learning is split into the following phases:

1. Fix a single minibatch of images from the dataset. Train F_θ and D_φ on this minibatch until bit accuracy of m against $\hat{m}$ crosses t_1 . Sample m 's bits uniformly and randomly. Before passing $\tilde{c}$ into D_φ , we choose with probability $1/2$ whether to crop $\tilde{c}$, in which case we crop $\tilde{c}$ to a random patch with sides of at least $3P$ in length.
2. Expose F_θ and D_φ to a much larger portion of the training set (50,000 images in our case). Train until bit accuracy reaches t_2 as this new exposure will likely drop the bit accuracy by a significant amount.
3. Begin incrementing β linearly by some amount δ_β until it reaches some β_{max} , at which point continue training until bit accuracy reaches t_3 .
4. Reveal the full dataset to F_θ and D_φ and train until bit accuracy reaches t_4 .
5. Introduce a noise function N which manipulates $\tilde{c}$ to generate a noised image $N(\tilde{c})$. Pass $N(\tilde{c})$ into D_φ . Train for an additional k epochs.

4 Implementation Details

For simplicity, we trained on square images of size 384×384 using a patch size of $P = 96$. All of our experiments are done with a message length of $\ell = 20$. We picked $\alpha = 1.5$ as in RoSteALS. We chose $\beta = 0.8$ and increment β linearly until $\beta_{\text{max}} = 11$ in training phase 3. The bit accuracy thresholds are set to $t_1 = 0.9, t_2 = 0.8, t_3 = 0.95, t_4 = 0.98$. We used the first 100,000 images from the COCO training dataset from 2017. We optimize the objective in (3) using stochastic gradient descent (in PyTorch), a minibatch size of 8, and the AdamW optimizer with a learning rate of $2 \cdot 10^{-5}$.

We use a very small neural network for F_θ with only 9,168 parameters. The network is described by the following PyTorch Sequential network:

```
nn.Linear(20, 12 * 12 * 3),
nn.SiLU(),
nn.Unflatten(1, (3, 12, 12)),
nn.Upsample(scale_factor=2),
nn.Conv2d(3, 3, 3, padding = 1),
nn.Conv2d(3, 3, 1)
```

where, as in RoSteALS, the final layer is 0-initialized so that $\delta = 0$ at the start training.

5 Experiments

TODO fill in with info about the experiments.

6 Results

TODO fill in with photos of the images.

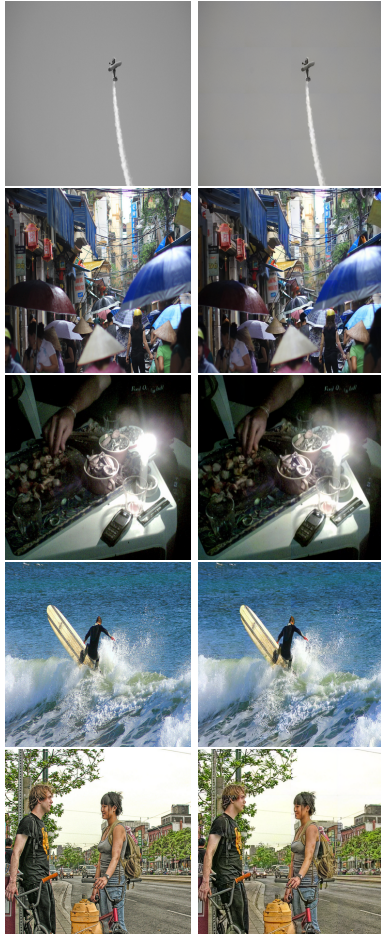


Figure 2: Cover images (*left*) and their watermarked counterparts (*right*) produced by StegoPatch. Early experiments, light segmenting can still be seen in the airplane (top) and the pair talking (bottom).